

Data Engineering Course - roadmap i narzedzia

2026-07-12

Course intro

■ Programmer -> Data Engineer

▣ Roadmapa, sposob pracy i narzedzia

Cel pierwszego spotkania: zobaczyc cala mape, zanim zaczniemy pisac SQL.

Nie musisz znac tych pojec perfekcyjnie. Masz miec intuicje, zeby slajdy nie byly lista obcych slow.

```
Production DB = baza aplikacji, np. zamowienia i platnosci
Data lake = tanie miejsce na pliki i surowe dane
Warehouse = miejsce pod szybkie SQL, raporty i analityke
Lakehouse = data lake + tabele + transakcje + SQL
ETL = czyscimy dane przed zaladowaniem
ELT = ladujemy dane najpierw, czyscimy pozniej w
warehouse/lakehouse
Bronze = dane blisko zrodla
Silver = dane wyczyszczone
Gold = dane gotowe pod metryke, dashboard albo raport
```

Jesli nie rozumiesz slowa, zatrzymaj slajd i dopisz pytanie.

■ Po co ten kurs?

Nie uczymy listy narzędzi.

Uczymy się budować zaufany przepływ danych:

- skąd dane pochodzą,
- co oznacza jeden rekord,
- jak dane są czyszczone,
- kto ich używa,
- jak sprawdzamy, że wynik jest poprawny,
- co robimy, gdy pipeline padnie.

Co robi DE na co dzien:

```
BUILD    projektuje i buduje pipeline: source -> Bronze -> Silver  
-> Gold  
TRUST    sprawdza ze dane sa kompletne, poprawne i na czas  
FIX      debuguje co padlo, szuka pierwotnej przyczyny, naprawia  
SHIP     wersjonuje, testuje, dokumentuje jak software engineer  
EXPLAIN  tłumaczy analitykowi i biznesowi co w danych i czemu
```

Co musi umiec:

- SQL: CTE, window functions, EXPLAIN, grain, kardynalnosc
- Python: ekstrakcja, walidacja, orkestracja (Airflow DAG)
- model danych: relacje, klucze, typy, NULL semantics
- pisac idempotentne pipeline – run N razy = ten sam wynik
- data quality checks – blad widoczny natychmiast, nie ukryty w raporcie
- git + PR + code review jak software engineer
- czytac logi i debugowac bez paniki o 3 w nocy
- tłumaczyc trade-off: batch vs stream, lake vs warehouse, cost vs latency

■ Docelowa umiejetnosc

Po kursie umiesz zbudowac, wyslac i wyjasnic produkcyjny pipeline od zera.

SQL

- pisac zapytania z JOINami, CTE, funkcjami okienkowymi
- definiowac grain i udowodnic, ze go nie lamiesz
- debugowac duplikaty i NULL-e bez podpowiedzi
- czytac **EXPLAIN** i powiedziec, gdzie jest bottleneck

Pipeline i architektura

- zaprojektowac Medallion: Bronze → Silver → Gold z uzasadnieniem
- napisac idempotentny krok ETL (uruchom dwa razy, wynik ten sam)
- dodac data quality check, który zatrzyma pipeline przy zlych danych
- zaplanowac zadanie w Airflow (DAG, task, dependency, retry)

Modelowanie

- opisac model wymiaru i model faktow (star schema)
- dobrac typ join i uzasadnic decyzje
- przetransformowac surowy flat-file w analytical mart

Narzedzia

- pisac kod SQL w SQLite, DuckDB, Postgres (te same wzorce)
- korzystac z dbt: model, test, dokumentacja
- wersjonowac pipeline w git (branch, PR, review)
- czytac logi i naprawic failing task samodzielnie

Komunikacja i interview

- wyjasnic swoj pipeline niedevcowi w 2 minuty
- napisac README i runbook dla on-call
- opowiedziec o kompromisie technicznym (np. batch vs stream)
- odpowiedziec na pytanie: "co by sie stalo, gdyby to zadanie padlo o 3 w nocy?"

■ Najważniejsza zmiana mindsetu

Backend:

request -> service -> database -> response

Data Engineering:

source -> ingestion -> transform -> model -> validate -> serve

Backend pyta: czy request działa teraz?

Data Engineer pyta: czy wynik jest poprawny, powtarzalny i zrozumiały jutro?

■ Dwie sciezki danych

BATCH / transakcje

App -> Production DB -> snapshot / CDC -> Bronze -> Silver -> Gold

STREAM / eventy

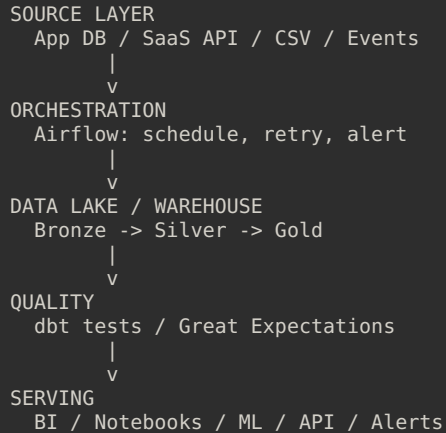
App -> Kafka / PubSub -> consumer -> Bronze -> Silver -> Gold

Gold -> Dashboards / ML / Alerts

Event = fakt ze cos sie stalo: order_created, payment_failed, page_view. Obie sciezki laduja do tych samych warstw Medallion.

```
Source DB / API / Files / Kafka
|
v
[ ingestion / extraction ]
|
v
BRONZE  raw data, blisko zrodla
|
v
SILVER  clean data, typy, dedup, joiny
|
v
GOLD    marts, metryki, raporty
|
v
Dashboards / ML / Reports
```

Ten slajd jest mapa rozmowy: source, ingestion, Bronze, Silver, Gold, quality i odbiorcy.



■ Ten sam flow jako model mentalny

Diagram warto umieć przerysować prosto:

```
Application
| writes transactions
v
Production DB -- daily snapshots / CDC --> Data Lake / Bronze
Application
| emits events
v
Kafka / PubSub -----> Data Lake / Bronze
Data Lake / Bronze
-> Silver / master data
-> Gold marts / OLAP cubes
-> Metrics / dashboards / decisions
```

Najważniejsze:

- aplikacja tworzy dane,
- metryka powstaje po czyszczeniu i modelowaniu.

Różnica jest w miejscu transformacji:

ETL:

Extract -> Transform -> Load

dane czyszcimy/przekształcamy przed załadowaniem do docelowej warstwy

ELT:

Extract -> Load -> Transform

dane najpierw ładujemy do lake/warehouse, potem transformujemy SQL/dbt/Spark

Dlaczego ELT wygrywa w nowoczesnym DE:

- surowe dane zostają w lake – możesz przetransformować ponownie bez powrotu do źródła,
- warehouse/lakehouse jest wystarczająco mocny, żeby robić transformacje SQL na skali,
- zmiana logiki biznesowej = tylko re-run transformacji, nie dotykasz extraction,
- dbt robi "T" deklaratywnie: testowalny, wersjonowany, z lineage.

Jednym zdaniem: osobny system/baza do analityki – nie baza aplikacji.

Najprosciej:

```
Production DB = baza aplikacji: szybkie zapisy pojedynczych  
zamowien  
Data Warehouse = baza analityczna: szybkie SQL po wielu rekordach  
naraz
```

To nie jest "skupisko baz danych". To zwykle jeden docelowy system analityczny. Kopiujemy i porzadkujemy w nim dane z wielu zrodel.

Analogia: biblioteka z katalogiem. Zanim cos wlozysz, musisz to opisac i skatalogowac.

Kiedy: ustrukturyzowane dane, duzo zapytan BI, stabilny schemat, szybkie agregaty.

Ryzyko: drogi na surowe dane, zmiana schematu bolesna, nie nadaje sie na logi i pliki binarne.

Architektura 2: Data Lake

Jednym zdaniem: tanie miejsce na pliki – surowe, nieprzetworzone, w kazdym formacie.

Analogia: tani magazyn. Wrzucasz wszystko celowo, zeby przetworzyc pozniej. Bez organizacji nikt nie wie, co tam jest.

Technologie: S3 + Parquet + Spark (AWS), GCS + BigQuery External, ADLS + Databricks

Kiedy: ogromne wolumeny, surowe dane do ML, logi, historia, tanie przechowywanie.

Ryzyko: bez governance lake zamienia sie w data swamp. Nikt nie wie, co jest w srodku, kto jest wlascicielem i czy dane sa swieze.

Lake ma surowe logi z 3 lat.

Jak znajdziesz event order_created, jesli nie ma schematu ani wlasciciela?

■ Architektura 3: Lakehouse

Jednym zdaniem: data lake z gwarancjami SQL.

Dodaje ACID, schemat i time travel na plikach Parquet.

```
Problem lake:  
  tani storage, ale malo gwarancji tabeli  
  
Table formats dodaja:  
  ACID, schema evolution, time travel, upserts
```

Technologie:

- Databricks + Delta Lake,
- Apache Iceberg + Trino/Spark.

Kiedy: chcesz taniego storage lake + SQL jak warehouse + dostep dla ML.

Ryzyko: bardziej zlozony setup i operacje niz czysty warehouse.

Jednym zdaniem:

Batch = przetwarzaj dane paczkami w oknach czasu. Streaming = przetwarzaj zdarzenia od razu, gdy przychodzą.

BATCH

co wiemy po oknie czasu?
daily/hourly job
Airflow + Spark + SQL
Streaming
łatwiejszy debug i replay
(kolejność, late data)
większość DE pracy dzisiaj
real-time ML

STREAMING

co dzieje się teraz?
sub-sekunda latency
Kafka + Flink / Spark

trudniejszy debug

alerting, fraud detection,

Dlaczego batch najpierw:

- łatwiej kontrolować grain, joiny, walidacje i retry,
- streaming dodaje late data, ordering i exactly-once.

■ Batch vs Streaming: pytanie

Klient chce real-time dashboard z revenue.

Czy potrzebuje streaming, czy wystarczy batch co 5 minut?

■ Dlaczego produkcyjna baza nie wystarczy?

Production DB jest dla aplikacji:

- szybki zapis jednego zamówienia,
- aktualny stan systemu,
- relacje pod transakcje,
- minimalna duplikacja.

Analytics potrzebuje czego innego:

- historia,
- agregaty,
- stabilne metryki,
- tanie skany wielu rekordów,
- dane z wielu systemów.

■ Słownik na start

Pojecie	Najprosciej
OLTP	baza aplikacji, szybkie transakcje
OLAP	analitka, agregaty, raporty
Data lake	tanie miejsce na pliki i surowe dane
Warehouse	miejsce do szybkiego SQL/analitki
Lakehouse	lake + transakcje + metadata + SQL
Medallion	Bronze/Silver/Gold jako warstwy jakosci

```
Bronze
  dane blisko zrodla, mozna odtworzyc pipeline

Silver
  dane wyczyszczone, ujednolicone, z jasnym grain

Gold
  dane pod konkretny use case: dashboard, metryka, raport
```

Bronze trzyma surowe dane blisko zrodla. Mozesz odtworzyc pipeline, jesli cos sie zepsuje.

Gold musi miec grain, definicje metryki i checki. Odbiorca nie powinien zgadywac, co oznacza jeden rekord.

■ Co to znaczy grain?

Grain = odpowiedz na pytanie:

Co oznacza jeden rekord w tej tabeli?

Przykład:

- `customers`: jeden rekord = jeden klient,
- `orders`: jeden rekord = jedno zamówienie,
- `order_items`: jeden rekord = jedna pozycja zamówienia,
- `daily_sales`: jeden rekord = jeden dzień sprzedaży.

■ Czego używamy na zajęciach?

Minimalny toolchain:

- Git + GitHub/GitLab style PR thinking,
- VS Code,
- terminal Linux albo WSL Ubuntu,
- SQL client,
- SQLite na pierwsze SQL basics,
- DuckDB po fundamentach SQL,
- Python 3.11+,
- pytest,
- Docker,
- dbt,
- Airflow lokalnie,
- Markdown docs.

■ Dlaczego SQLite teraz, DuckDB potem?

SQLite:

- dobry do absolutnych podstaw SQL,
- zero server setup,
- idealny na mały kontrolowany schema/seed,
- pomaga skupić się na grain, joinach, agregacji i walidacji.

DuckDB:

- wchodzi po fundamentach,
- szybki lokalny OLAP,
- czyta CSV/Parquet bez serwera,
- pasuje do labów data engineering bez ciężkiej infrastruktury.

Najpierw rozumiemy mały model danych. Potem bierzemy większe pliki i DuckDB.

■ Czego NIE robi Postgres w tej sciezce?

Postgres nie jest finalnym OLAP celem kursu.

Jest dobry jako:

- baza aplikacyjna,
- system zrodlowy OLTP,
- przyklad snapshot/CDC source,
- kontekst integracji w pozniejszych modulach.

Analityczny lokalny tor: DuckDB + CSV/Parquet.

■ Jak pracujemy co tydzień?

1. First principles: jaki problem rozwiązujemy?
2. Minimalny przykład.
3. Live lab.
4. Artefakt w repo.
5. Review checklist.
6. Krótka odpowiedź interview.

Każda lekcja zostawia ślad w repo.

Materialy lekcji maja taki uklad:

```
lesson_XX/
└─ student/
    └─ README.md          <- startuj tutaj, otwierasz jako
pierwsze
    └─ teoria.md          <- teoria, wzorce, pytania
sprawdzajace
    └─ slides.presenterm.md <- slajdy z zajec (do obejrzenia
po)
    └─ lab/
        └─ ...            <- aktywne cwiczenia do utrwalenia
```

Lekcja 00 ma dodatkowo `setup.md` i lab `data flow workshop.md`.

Od Lesson 01 dochodzi osobny homework SQL.

Check: przed kazdymi zajecia otwierasz student/README.md jako pierwsze.

■ Miesiac 1

■ SQL i modelowanie danych

- Praca z terminalem i Git: lokalne repo, commit, branch, push, PR thinking
- SQL od podstaw do zaawansowanego: grain, JOIN, agregacje, CTE, window functions
- Modelowanie danych: Medallion, Kimball, OBT, data contracts
- Pierwsze artefakty: query z walidacja, grain map, data product brief, PR do review

Cel: umiesz napisac i wyjasnic zapytanie SQL od grain po metryke oraz obronic model danych.

■ Programowanie i ekosystem analityczny

- Programowanie od podstaw: typy, funkcje, petle, wyjatki, czysty kod
- Python i wizualizacja: pandas, polars, podstawowe wykresy jako narzedzie diagnostyki danych
- Srodowisko developera: Linux, Docker, repozytoria, virtualenv, debugger
- Projekt: maly ETL jako projekt inzynierski – extract, transform, validate, load, testy, CLI

Cel: umiesz napisac ETL jak projekt, nie jednorazowy skrypt. Twój kod jest czytelny, testowalny i uruchamialny w powtarzalnym srodowisku.

■ Big Data i chmura

- Przetwarzanie rozproszone: Spark/PySpark foundations, partycje, shuffle, plan wykonania
- Chmura: podstawy GCP/Azure, storage, compute, uprawnienia, koszty
- Platforma danych w stylu lakehouse: Delta/Iceberg, table formats, schemat, time travel
- Projekt: lokalny pipeline skaluje sie do chmury

Cel: rozumiesz jak lokalny ETL staje sie pipeline produkcyjnym na duzych danych.

■ Produkcyjne systemy danych

- Orkiestracja: Airflow – DAG, task, dependency, retry, alert, idempotency
- Warstwa transformacji i jakosci: dbt modele, testy, dokumentacja, lineage
- Streaming i architektura zdarzeniowa: Kafka basics, consumer/producer, late data, event schema
- Data quality i observability: Great Expectations, alerty, SLA

Cel: twój pipeline jest zaplanowany, obserwowalny i ma runbook na wypadek błędu o 3 w nocy.

■ Projekt koncowy i portfolio

- End-to-end system danych zbliżony do produkcyjnego: od źródła przez transformacje po monitoring
- Infrastruktura jako kod: podstawy IaC, reproducible setup
- Obrona projektu: tłumaczysz decyzje techniczne, trade-offy, co by się stało przy N=10x danych
- Przygotowanie portfolio: README, architektura, runbook, mock interview

```
source -> Bronze -> Silver -> Gold -> dbt -> Airflow -> monitoring  
-> README
```

Cel: masz projekt który mozesz pokazac na rozmowie i bronic kazdej decyzji.

■ Czego nie robimy na start?

Nie wchodzimy głęboko w:

- Spark internals,
- administracje Kafka,
- Terraform,
- Unity Catalog,
- pełny Iceberg lab,
- Data Vault jako osobny blok.

Te tematy mają sens dopiero, gdy rozumiemy data flow, grain i reliable batch.

■ Dzisiaj ma zostac w glowie

Data Engineer nie buduje tylko tabel.

Buduje przepływ danych, któremu można zaufać.

Jesli nie wiesz, co oznacza jeden rekord, nie wiesz jeszcze, co liczysz.

■ Mini-zadanie przed zamknięciem laptopa

Narysuj diagram data flow w Excalidraw:

```
https://excalidraw.com
```

Na diagramie umiesz wszystkie etapy:

```
Application -> Production DB -> Bronze -> Silver -> Gold  
-> Metrics
```

Dla każdego etapu dodaj notkę:

- kto zapisuje dane,
- kto czyta dane,
- co może pójść źle,
- jak sprawdzisz poprawność.

Wpisz odpowiedzi w `student/lab/data flow workshop.md`. Jeśli robisz diagram w Excalidraw, dodaj link albo screenshot obok odpowiedzi.